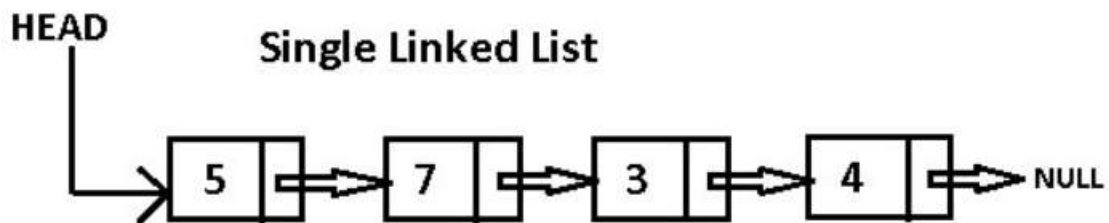
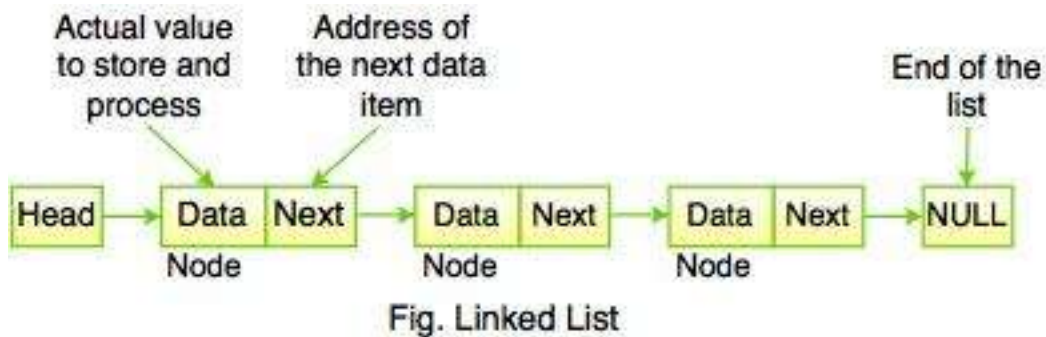


Lecture: 5

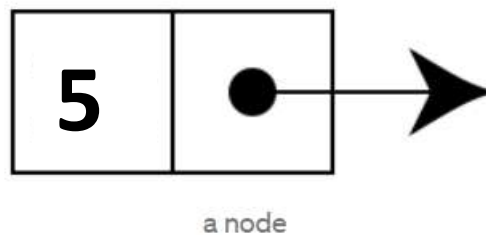
Linked Lists

A linked list is a data structure that consists of a sequence of data records such that in each record there is a field **that contains a reference (i.e. a link) to the next record in the sequence.**

i.e. **elements are not stored at contiguous memory locations in a linked list.** The elements in a linked list are linked using pointers.



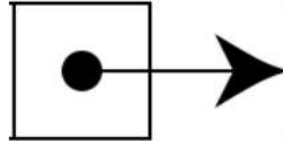
- A linked list consists of nodes:



- Each node consists of a value and a pointer to another node:



a value



a pointer

- The starting node of a linked list is referred to as the header.
- Last node references NULL. The NULL pointer denoted by x in the diagram, signals the end of the list.
 - In actual practice, 0 or a negative number is used for the null pointer.

Why use linked lists?

Linked list is often compared to arrays. Whereas an array is a fixed size of sequence, a linked list can have its elements to be dynamically allocated.

- **Advantages:**
 - A linked list **saves memory**. It only allocates the memory required for values to be stored.
 - In arrays, you have to set an array size before filling it with values, which can potentially waste memory.
 - Linked list nodes can live anywhere in the memory.
 - Whereas an array requires a sequence of memory to be initiated, as long as the references are updated, each linked list node can be flexibly moved to a different address.
- **Disadvantages:**
 - When looking for a value in a linked list, you have to start from the beginning of chain and check one element at a time for a value you are looking for. If the linked list is n elements long, this can take up to n time.

Representation of Linked List in Memory:

Let LIST be a linked list. Then LIST will be maintained in memory as follows:

- LIST requires two linear arrays:
 - INFO [K]: contains the information part of a node of a LIST.
 - LINK[K]: contains the nextpointer filed of a node of LIST.
- LIST also requires a variable name:
 - START: which contains the location of the beginning of the list
 - a nextpointer sentinel denoted by NULL (value=0), which indicates the end of the list.

Example: Fig below pictures a linked list in memory where each node of the list contains a single character:

